Introduction to Interpretability and Explainable Artificial Intelligence

Contents

1 Introduction to Interpretability and Explainable AI **2**
1.1 Shortcut Learning 2

2 Families of Explainability Methods **3**
2.1 White-box vs Black-box Methods 3
2.2 Local and Global Explanations 4
2.3 Main Categories of XAI Methods 4
2.3.1 Gradient-based Methods 4
2.3.2 Perturbation-based Methods 5
2.3.3 Attribution-based Methods 5
2.3.4 Concept-based Methods 5

3 Detailed examination on specific algorithms **5**
3.1 LIME (Local Interpretable Model-agnostic Explanations) 5
3.2 SHAP (SHapley Additive exPlanations) 6
3.3 Grad-CAM 7

1 Introduction to Interpretability and Explainable AI

Explainable Artificial Intelligence (XAI) refers to a set of methods and techniques designed to make the decisions of machine learning models more understandable to humans. In recent years, highly complex models such as deep neural networks have achieved remarkable performance in tasks such as image recognition, natural language processing, and medical diagnosis. However, these models often operate as "black boxes": while they produce highly accurate predictions, the reasoning behind their decisions is not easily inspectable.

Before proceeding, it is worth distinguishing two related but distinct concepts. *Interpretability* is an intrinsic property of a model, a model is interpretable if its structure can be understood directly, without external tools (e.g. linear regression, decision trees). *Explainability*, by contrast, refers to the ability to produce a post-hoc description of a black-box model's behavior through a separate method. Techniques such as SHAP, LIME, and Grad-CAM are explainability tools. In practice, the term XAI is used as an umbrella covering both.

The motivations behind XAI are several. The lack of transparency raises important concerns in domains such as healthcare, finance, and autonomous systems, where understanding the reasoning behind a model's prediction is crucial for trust, accountability, and safety. Beyond compliance, interpretability supports debugging: knowing which features drive a prediction helps to identify and fix data issues or modelling errors. It also enables scientific discovery, as in medicine or biology where understanding the model's reasoning can yield genuine insight into the underlying system.

For instance, one failure mode that XAI helps to detect is shortcut learning: models may rely on unintended correlations present in the training data rather than the true underlying patterns. Such models can achieve high benchmark accuracy while having learned entirely the wrong features, and fail silently when deployed in the real world. This is one of the several reasons to adopt interpretability tools in practice, and it is examined in detail in the next section.

1.1 Shortcut Learning

Shortcut learning (also called *Clever Hans behavior* or *spurious correlation learning*) occurs when a model minimizes training loss by relying on features that are irrelevant to the true task, rather than learning the intended underlying patterns. A model trained this way can achieve near-perfect accuracy on benchmark data and then fail completely when deployed in the real world.

The name comes from a famous historical episode. In the early 1900s, a horse named Clever Hans appeared to solve arithmetic problems. However, investiga-

tions revealed the horse was not calculating at all: it was reading involuntary body-language cues from its trainer, who was entirely unaware of providing them. Neural networks exhibit precisely the same behavior: they exploit whatever signal minimizes the loss, regardless of whether that signal is meaningful.

We can then define a **shortcut feature** as one that is highly correlated with the label *in training data* but uncorrelated, or even negatively correlated, at deployment time. Concretely, if we add a spurious feature s that is 99% correlated with y during training but drawn independently from y at test time, a logistic regression will assign s the largest coefficient and essentially ignore the real signal features. The model will reach near-perfect training accuracy, then collapse to chance at test time. An example in real-world scenario is the case of the skin-cancer classifier where the model earned to detect ruler marks in dermoscopy images. Rulers happened to appear more frequently next to malignant lesions in the training set, so the model used them as a proxy for malignancy rather than inspecting the skin tissue itself.

In conclusion, high accuracy alone is therefore not a guarantee of correct reasoning. Interpretability and explainability methods allow us to understand model behavior before deployment, catching Clever Hans effects while there is still time to fix the data or the training procedure.

2 Families of Explainability Methods

2.1 White-box vs Black-box Methods

A fundamental distinction in XAI concerns the degree of access we have to the model's internals:

White-box models are those whose structure and parameters are fully accessible and human-readable. Linear regression, logistic regression, and decision trees are canonical examples: the weights or decision rules can be inspected directly to understand why the model produces a given output. Interpretability here is *intrinsic* and no additional tool is needed. The trade-off is that these models are typically less powerful than deep neural networks on complex tasks.

Black-box models, by contrast, expose only their input–output behaviour: we can query them and observe predictions, but the internal computation is either inaccessible (proprietary APIs) or too complex to reason about directly (deep neural networks with millions of parameters). Most state-of-the-art models in computer vision and natural language processing fall into this category. Explainability methods are applied *on top* of these models to recover post-hoc explanations.

It is worth noting that the white-box/black-box distinction is not always bi-

nary. Some models, such as shallow neural networks or regularised ensembles, occupy an intermediate space, and techniques like attention visualization offer partial transparency within an otherwise opaque architecture. The choice between intrinsic interpretability and post-hoc explainability involves a practical trade-off: simpler, interpretable models are easier to audit but may sacrifice predictive performance, while black-box models require an additional explanation layer that introduces its own approximations.

2.2 Local and Global Explanations

A fundamental dimension along which explainability methods differ is their *scope*: whether they explain a single prediction or the model's overall behavior.

Local explanations target one specific instance. Given a particular input, a local method asks: "Why did the model produce *this* output for *this* input?" The explanation is valid only in the neighborhood of that data point and says nothing about the model elsewhere. LIME, for example, fits a simple linear surrogate around a single input by perturbing it and querying the model. The resulting coefficients explain that one prediction. Grad-CAM similarly produces a per-image heatmap that highlights the regions driving a specific classification.

Global explanations describe the model's behavior across the entire input distribution. They ask: "What patterns has the model learned in general?" Rather than zooming in on one case, global methods summarize how the model uses features on average. For instance, SHAP beeswarm and bar plots aggregate per-sample attribution scores over many data points, revealing which features matter most overall.

The two scopes are complementary rather than competing. Some methods naturally operate at both levels: SHAP, for instance, computes a precise local attribution for each individual prediction and then aggregates those scores into global summary statistics. In practice, a thorough interpretability analysis combines both local and global perspectives: local explanations catch individual failures or edge cases, while global explanations reveal whether the model has learned sensible patterns across the board.

2.3 Main Categories of XAI Methods

Finally, post-hoc explainability methods can be grouped into four broad families depending on how they probe the model's behavior.

2.3.1 Gradient-based Methods

Gradient-based methods exploit the fact that, for differentiable models such as neural networks, the gradient of the output with respect to the input directly quantifies how sensitively the prediction responds to each input dimension. The

core algorithms in this family are *Vanilla Gradient* [7], *Guided Backpropagation* [8], *Integrated Gradients* [9], *SmoothGrad* [10], and *Grad-CAM* [4]. Unlike perturbation-based approaches, these methods require white-box access to the model and produce explanations by performing one or more backward passes through the network.

2.3.2 Perturbation-based Methods

Rather than differentiating through the model, perturbation-based methods treat it as a black box and measure how predictions change when parts of the input are masked or modified. The main algorithms in this family are *LIME* [1] and *Occlusion-based explanations*.

2.3.3 Attribution-based Methods

Attribution-based methods assign a numerical importance score to each input feature such that the scores account for the total prediction. The key algorithms are *SHAP* [2], grounded in cooperative game theory, and *DeepLIFT* [3].

2.3.4 Concept-based Methods

Concept-based methods move beyond raw input features and ask which high-level, human-defined concepts the model relies on internally. Representative algorithms are *Probing classifiers* [6] and *TCAV* [5].

3 Detailed examination on specific algorithms

In this section we are going to examine the main algorithms in explainability.

3.1 LIME (Local Interpretable Model-agnostic Explanations)

LIME [1] is a **model-agnostic**, **local** explainability method. Its core insight is that even if a model is globally complex, it is possible to approximate its behavior *locally*, in the neighborhood of a specific input, with a simple, interpretable surrogate model such as a linear regression or a decision tree.

The algorithm proceeds in three steps. First, given the input x to be explained, LIME generates a set of *perturbed* samples $\{z_1, \ldots, z_n\}$ by randomly masking or modifying parts of the input (e.g. hiding image patches, removing words from a sentence, or perturbing tabular feature values). Second, it queries the black-box model f on all perturbed samples to obtain predictions $f(z_i)$, and weights each sample by its proximity to the original input x using a kernel function $\pi_x(z_i)$. Third, it fits a simple interpretable model g (typically linear) by solving:

$$g^* = \arg\min_{g \in \mathcal{G}} \sum_i \pi_x(z_i) \left(f(z_i) - g(z_i)\right)^2 + \Omega(g) \tag{1}$$

where $\Omega(g)$ is a regularization term that penalizes model complexity. The coefficients of g^* are then returned as the explanation: large positive coefficients indicate features that push the prediction toward the target class, while large negative coefficients indicate features that push against it.

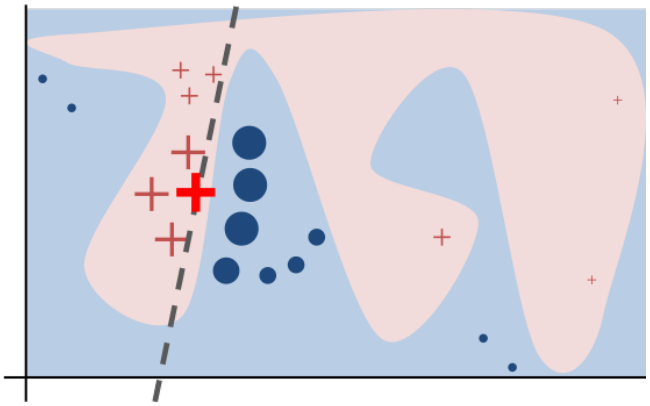

Figure 1: Toy example to present intuition for LIME. The black-box model's complex decision function f (unknown to LIME) is represented by the blue/pink background, which cannot be approximated well by a linear model. The bold red cross is the instance being explained. LIME samples instances, gets predictions using f, and weighs them by the proximity to the instance being explained (represented here by size). The dashed line is the learned explanation that is locally (but not globally) faithful. (source: [1])

A key strength of LIME is its **model-agnosticism**: it works with any classifier or regressor, whether a neural network, a random forest, or a proprietary API (like ChatGPT), since it only requires the ability to query the model. It is also applicable across data modalities (images, text, tabular data) with appropriate perturbation strategies. The main limitation is that the explanation is only locally faithful: the linear surrogate may be a poor approximation far from x, and the quality of the explanation depends on the perturbation strategy and the choice of kernel bandwidth. In practice, LIME explanations can be unstable, which means that running it twice on the same input may yield different feature attributions. For a more in-depth treatment we recommend LIME [11].

3.2 SHAP (SHapley Additive exPlanations)

SHAP [2] is an **attribution-based** method grounded in cooperative game theory. It provides a principled way to assign an importance score, called a *Shapley value*, to each input feature, measuring how much each feature contributed to shifting the model's prediction away from a baseline (typically the average prediction over the training set).

The Shapley value originates from a classical problem in economics: how to fairly distribute the total payoff of a cooperative game among its players. In the XAI context, features are the "players" and the model's prediction is the "payoff". The Shapley value of feature i is defined as:

$$\phi_i = \sum_{S \subseteq \mathcal{F} \setminus \{i\}} \frac{|S|!\,(|\mathcal{F}| - |S| - 1)!}{|\mathcal{F}|!} \Big[f(S \cup \{i\}) - f(S) \Big] \tag{2}$$

where $\mathcal{F}$ is the set of all features and $f(S)$ denotes the model's expected prediction when only the features in S are known (all others are marginalized out). Intuitively, the Shapley value for feature i is its average marginal contribution across all possible subsets of the other features, a measure that accounts for feature interactions.

Practical variants. The exact computation of Shapley values requires evaluating the model on all $2^{|\mathcal{F}|}$ feature subsets, which is intractable for large feature sets. In practice, several efficient approximations exist. *KernelSHAP* uses a LIME-like sampling procedure with a specific kernel weighting that provably recovers Shapley values. *TreeSHAP* is an exact, polynomial-time algorithm for tree-based models (random forests, gradient boosting) that exploits the tree structure to compute values without exponential enumeration.

Local and global use. At the local level, a SHAP force plot or waterfall chart shows for a single prediction how each feature pushes the output above or below the baseline. At the global level, aggregating Shapley values over many data points yields summary statistics: beeswarm plots display the distribution of each feature's attribution across the dataset, while bar charts show mean absolute attributions as a global feature ranking. This dual nature makes SHAP one of the most widely used explainability tools in practice. For a more in-depth treatment we recommend SHAP [11]

3.3 Grad-CAM

Grad-CAM [4], short for *Gradient-weighted Class Activation Mapping*, is a **local**, **gradient-based**, **CNN-specific** explainability method that produces a coarse spatial heatmap highlighting the image regions most influential for a given class prediction. Unlike model-agnostic methods such as LIME or SHAP, Grad-CAM is specifically designed for convolutional neural networks (CNNs) and exploits the gradient signal that flows through the network during a backward pass. Its core insight is that the feature maps of the last convolutional layer retain spatial information that has been progressively abstracted by the network, and that the gradients of the class score with respect to those feature maps encode which spatial locations matter most for the prediction of that particular class. By combining these two sources of information, Grad-CAM can localize discriminative image regions without requiring any modification to the network architecture or retraining.

The algorithm proceeds as follows. Let y^c denote the score (logit) for class c before the softmax, and let $A^k \in \mathbb{R}^{H \times W}$ denote the k-th feature map of the last convolutional layer, where H and W are the spatial dimensions. Grad-CAM computes the gradient of y^c with respect to every spatial position (i, j) of every feature map A^k, then globally average-pools these gradients to obtain a scalar weight α_k^c for each feature map:

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k} \tag{3}$$

where $Z = H \times W$ is the total number of spatial positions. Intuitively, α_k^c measures the global importance of feature map k for class c. Grad-CAM then computes a weighted combination of all feature maps and applies a ReLU to retain only the locations with a positive influence on the class score:

$$L_{\text{Grad-CAM}}^c = \text{ReLU}\left(\sum_k \alpha_k^c \, A^k \right) \tag{4}$$

The ReLU is essential: without it, negative contributions from other classes would dilute class discriminability. The resulting localization map $L_{\text{Grad-CAM}}^c \in \mathbb{R}^{H \times W}$ is bilinearly upsampled to the original input resolution and overlaid as a heatmap.

The output of Grad-CAM is a **class-discriminative localization map**: regions colored with high intensity contributed positively to the prediction of class c. Crucially, running Grad-CAM for two different classes on the same image yields two distinct heatmaps, reflecting the network's distinct reasoning for each class.

The principal **strengths** of Grad-CAM are its architectural flexibility, its class-discriminativeness, and the fact that it requires no retraining or modification of the original model. A single forward and backward pass suffices to produce an explanation, making it computationally cheap relative to perturbation-based alternatives. Its main **limitations** are resolution and depth: the localization map inherits the spatial resolution of the last convolutional layer, which is typically much smaller than the input image, so the heatmap is inherently coarse.

Practical value. A primary use of Grad-CAM is to verify whether a model attends to the *correct* image regions rather than exploiting spurious shortcuts. Figure 2 illustrates this: a biased model focuses on the person's face (red box) when predicting "nurse" or "doctor", while an unbiased model correctly attends to the stethoscope (green box). Both achieve the same prediction, yet their heatmaps reveal fundamentally different reasoning strategies.

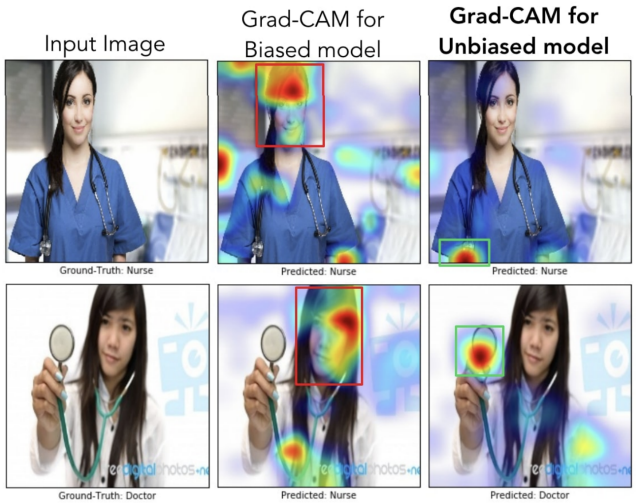

Figure 2: Grad-CAM localization maps for a biased model (center column) and an unbiased model (right column) [4]. The biased model focuses on the face (red box), while the unbiased model correctly attends to the stethoscope (green box).

References

[1] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why Should I Trust You?': Explaining the Predictions of Any Classifier," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 1135–1144, 2016.

[2] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 4765–4774, 2017.

[3] A. Shrikumar, P. Greenside, and A. Kundaje, "Learning Important Features Through Propagating Activation Differences," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pp. 3145–3153, 2017.

[4] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 618–626, 2017.

[5] B. Kim, M. Wattenberg, J. Gilmer, C. Cai, J. Wexler, F. Viégas, and R. Sayres, "Interpretability Beyond Classification with Quantitative Testing with Concept Activation Vectors (TCAV)," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pp. 2668–2677, 2018.

[6] G. Alain and Y. Bengio, "Understanding Intermediate Layers Using Linear Classifier Probes," *arXiv preprint arXiv:1610.01644*, 2016.

[7] K. Simonyan, A. Vedaldi, and A. Zisserman, "Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps," *arXiv preprint arXiv:1312.6034*, 2014.

[8] J. T. Springenberg, A. Dosovitskiy, T. Brox, and M. Riedmiller, "Striving for Simplicity: The All Convolutional Net," *arXiv preprint arXiv:1412.6806*, 2015.

[9] M. Sundararajan, A. Taly, and Q. Yan, "Axiomatic Attribution for Deep Networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pp. 3319–3328, 2017.

[10] D. Smilkov, N. Thorat, B. Kim, F. Viégas, and M. Wattenberg, "SmoothGrad: removing noise by adding noise," *arXiv preprint arXiv:1706.03825*, 2017.

[11] *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable Shap*, 2nd ed., 2022.